CS 739 MadKV Project 2

Group members: Karthik Reddy Jannupalli `jannupalli@wisc.edu`, Shivam Mittal `smittal39@wisc.edu`

Design Walkthrough

Architecture

Our Project 2 implementation consists of three components: **manager**, **server**, and **client**. The manager maintains cluster membership; each server owns one partition and stores data durably; the client routes requests to partitions using key hashing.

Manager

The manager listens on a configurable port and is given a comma-separated list of expected server endpoints at startup. It tracks which servers have registered via `RegisterServer`. Once all expected servers have registered, `GetCluster` returns `ready=true` and the partition list (server API addresses). Clients use this to build per-partition gRPC stubs. The manager uses the configured endpoints for `GetCluster` (not the listen address reported at registration) so clients can reach servers over the private network.

Server (WAL-Based State Machine)

Each server is a **WAL-based state machine** per spec: the source of truth is an in-memory `std::map`, and durability comes from a write-ahead log. On startup, the server replays the WAL to reconstruct state.

WAL format: Binary entries `[op:1][klen:4][vlen:4][key][value]` where `op` is SET (0) or DEL (1). Entries are appended and flushed with `fdatasync`.

Group commit: To avoid syncing on every write, we batch entries. A background flush thread wakes every 20ms or when 512 entries accumulate, then does a single `write()` + `fdatasync()` for the batch. Appenders wait until their sequence number has been flushed. This reduces disk syncs and improves throughput for write-heavy workloads.

Concurrency: The server uses `std::shared_mutex` for state. Put uses a lock-release-append-reacquire pattern to allow batching; Swap and Delete hold the lock for the full operation to preserve atomic read-modify-write semantics.

Client (Partitioned)

The client connects to the manager and fetches the partition list via `GetCluster`. It uses **FNV-1a 64-bit** to hash each key and selects the partition with `hash(key) % num_partitions`. Put, Get, Swap, and Delete are routed to the owning partition; Scan is sent to all partitions, and results are merged and sorted. On `UNAVAILABLE`, the client refreshes the cluster and retries. RPC deadlines are set to 300 seconds for long-running benchmarks.

Key Design Choices

- **Partitioning**: Hash-based partitioning distributes keys across servers. No replication; each key has one owner.
- **Durability**: File-based WAL with group commit instead of per-operation sync.
- **Recovery**: On restart, the server replays the WAL sequentially to rebuild state.

Self-provided Testcase

Process

The testcase validates partition failure and recovery:

1. **Launch 3 partitions** – Start manager and three servers on ports 3777, 3778, 3779.
2. **PUT/SWAP and GET/SCAN** – Insert keys into each partition (k0→partition 0, k1→partition 1, k2→partition 2 via FNV-1a hash), swap k1, then verify GET and SCAN.
3. **Kill server for partition 1** – SIGKILL the process for partition 1.
4. **GET/SCAN on unaffected partition** – Requests to partition 0 succeed (k0, SCAN k0..k0).
5. **GET on failed partition** – Client connects directly to the dead server; GET k1 is expected to time out (6s).
6. **Restart failed server** – Start the server again; it replays the WAL and restores state.
7. **GET after recovery** – GET k1 must return the latest value (v1_new), confirming durability across crash and restart.

This demonstrates that (a) one partition failing does not block others, (b) requests to a failed partition time out, and (c) after restart, data is recovered from the WAL.

Fuzz Testing

Parsed the following fuzz testing results:

num_servers	crashing	outcome
3	no	PASSED
3	yes	PASSED
5	yes	PASSED

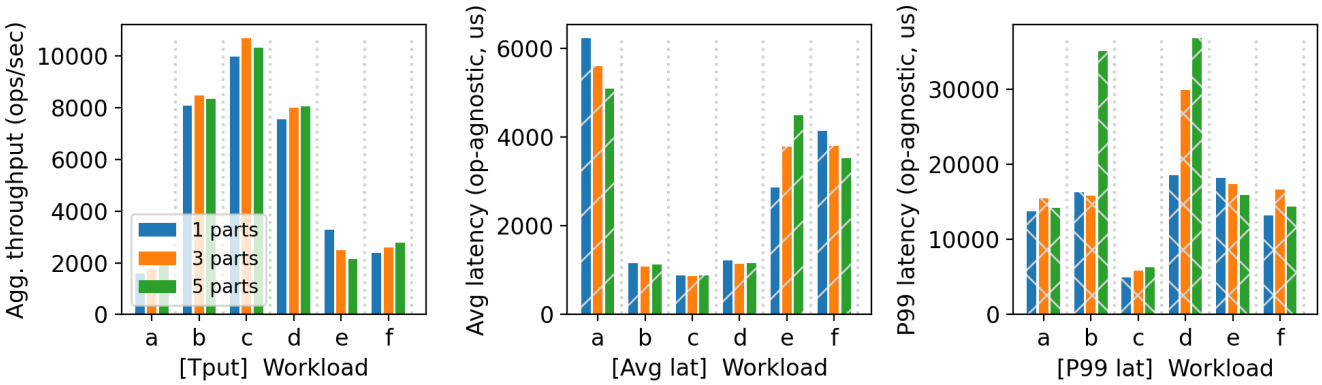
You will run a crashing/recovering fuzz test during demo time.

Comments

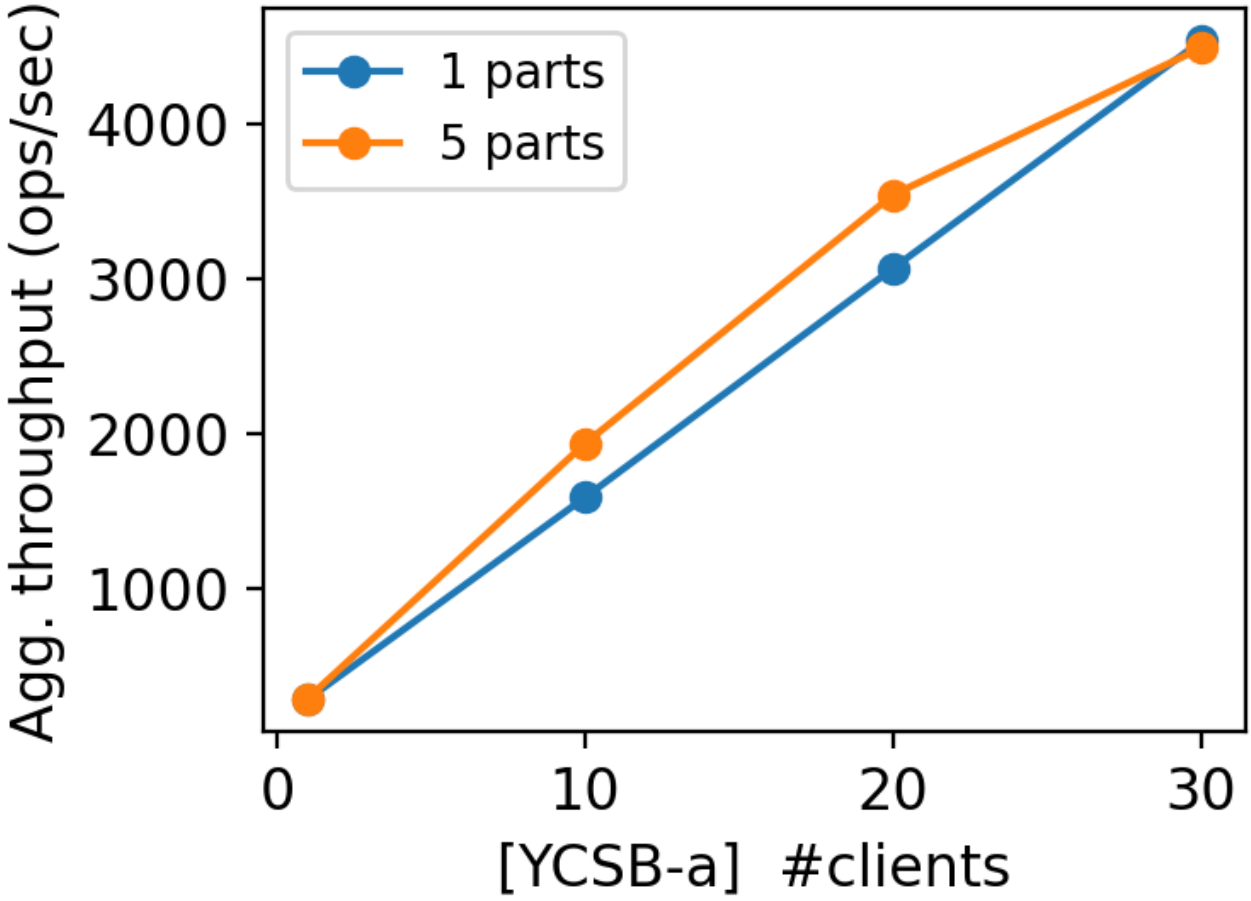
The fuzzer runs multiple clients issuing random Put, Swap, Get, Scan, and Delete operations with overlapping keys (conflict mode). It checks for linearizability using a causal-history model. All three configurations passed: healthy 3-partition, crashing 3-partition, and crashing 5-partition. The crashing tests periodically kill and restart servers; the fuzzer expects clients to handle timeouts and retry. Our client's cluster refresh on `UNAVAILABLE` and long RPC deadlines allow the fuzzer to complete without spurious failures.

YCSB Benchmarking

10 clients throughput/latency across workloads & number of partitions:



Agg. throughput trend vs. number of clients w/ and w/o partitioning:



Comments

Workload behavior: Workload C (read-only) typically shows the highest throughput and lowest latency because there are no writes. Workloads A and B include updates (Swap), which incur WAL appends and group-commit waits. Workloads D, E, and F add scan and read-modify-write patterns that increase load on partitions.

Partitioning: When partitions run on *different machines*, 5 partitions usually outperform 1 because work is spread across servers. When all partitions share the same machine, 1 partition can outperform 5 due to no process/context-switch overhead and simpler disk access. The throughput trend (1 vs 5 partitions) reflects whether partitioning adds real parallelism (separate machines) or extra overhead (single machine).

Latency: P99 latency is higher than average due to occasional slow syncs, lock contention, and tail latency in the network or disk. Write-heavy workloads tend to have higher P99.

Additional Discussion

Project Work Summary

Durability and storage

- Implemented a WAL-based state machine per spec: durable log of commands, in-memory `std::map`, replay on recovery. No direct RocksDB or file-based state persistence (we tried RocksDB with `sync=true` but it was too slow for the 600s YCSB timeout).
- Custom file WAL at `<backer_path>/wal.log` with binary entries `[op:1][klen:4][vlen:4][key][value]` (op = SET or DEL).
- Group commit: buffer up to 512 entries or flush every 20 ms; one `fdatasync` per batch instead of per operation. This required fixing a near-deadlock where all gRPC handler threads held the mutex while waiting for the flush thread; we switched to sequence counters so appenders release the lock during `write`/`fdatasync`.
- Put uses a lock-release-append-reacquire pattern so multiple Puts can batch; Swap and Delete hold the full lock for atomic read-modify-write semantics.

Client changes

- Raised RPC deadline from 2 s to 300 s to avoid benchmark timeouts.
- `RefreshClusterWithRetry` only on `UNAVAILABLE` (not on other errors such as `DEADLINE_EXCEEDED`), so clients recover from partition failures without unnecessary cluster refreshes.

Testing and deployment

- `run_p2_all.sh`: two-node script (node0 = client, node1 = server) for fuzz and bench.
- `run_p2_distributed.sh`: multi-node script with node0 = client, node1 = manager only, nodes 2–6 = servers; each partition on a separate machine. Requires passwordless SSH from node0 to nodes 1–6; private IPs configured for CloudLab.
- `run_p2_custom_testcase.sh`: deterministic crash-and-recovery testcase for demo.
- Report generation via `sumgen/proj2.py` (requires `matplotlib`, `termcolor`); produces `report/plots-p2/ycsb-ten-clients.png` and `ycsb-tput-trend.png`.

Lessons learned

- Partitioning improves throughput when partitions run on different machines; on a single machine, 1 partition can outperform 5 due to lower overhead.
- WAL group commit is essential for write-heavy workloads; per-operation sync is too slow.